

TITRE PROFESSIONNEL DÉVELOPPEUR WEB ET WEB MOBILE

ModHub

Plateforme communautaire de partage de mods pour jeux vidéo

[Prénom NOM] — [Centre de formation] — [Date]

Plan de la présentation

- Le candidat et le contexte du projet
- Le besoin et les objectifs de ModHub
- Démonstration de l'application
- Architecture technique et choix justifiés
- Focus compétences : front-end et back-end
- Sécurité et tests
- Difficultés rencontrées et bilan
- Questions

Qui suis-je ?

À compléter : parcours, formation, motivation pour le développement web

Pourquoi ce projet ?

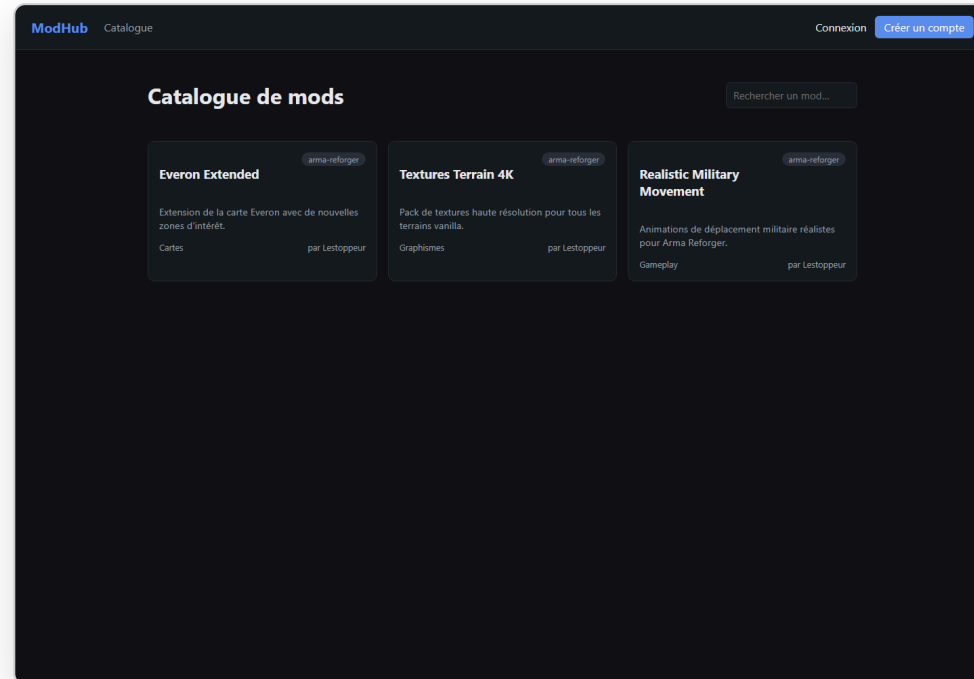
Les communautés de modding de jeux vidéo s'organisent souvent autour d'outils génériques (forums, Discord) mal adaptés au suivi structuré d'un mod : versions, compatibilité, retours qualitatifs.

ModHub centralise la publication, le suivi de versions et les retours communautaires autour des mods.

Publics et besoins

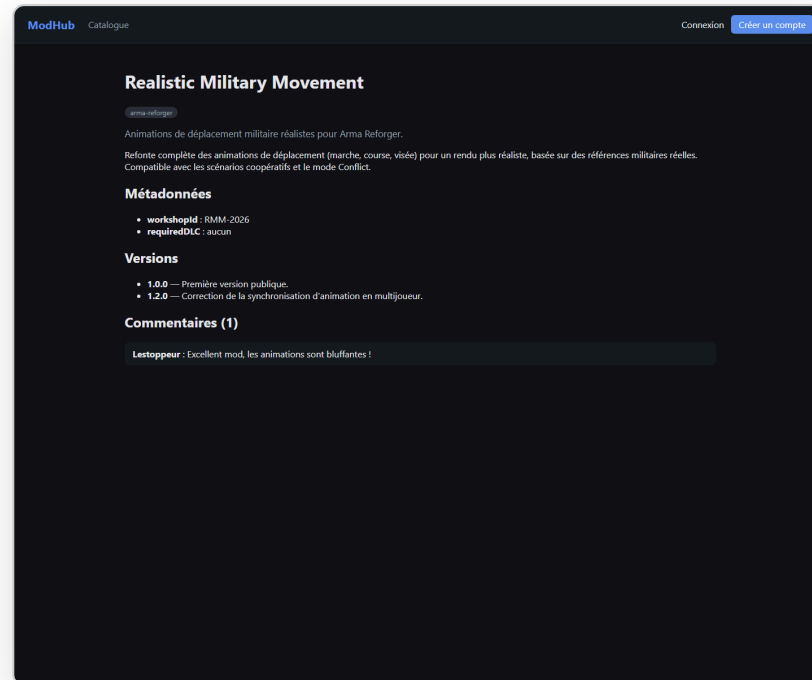
Public	Besoin
Créateurs	Publier, gérer des versions, suivre l'adoption
Communauté	Trouver, évaluer, suivre des mods de qualité
Modérateurs	Garantir la qualité du catalogue public

Le catalogue public



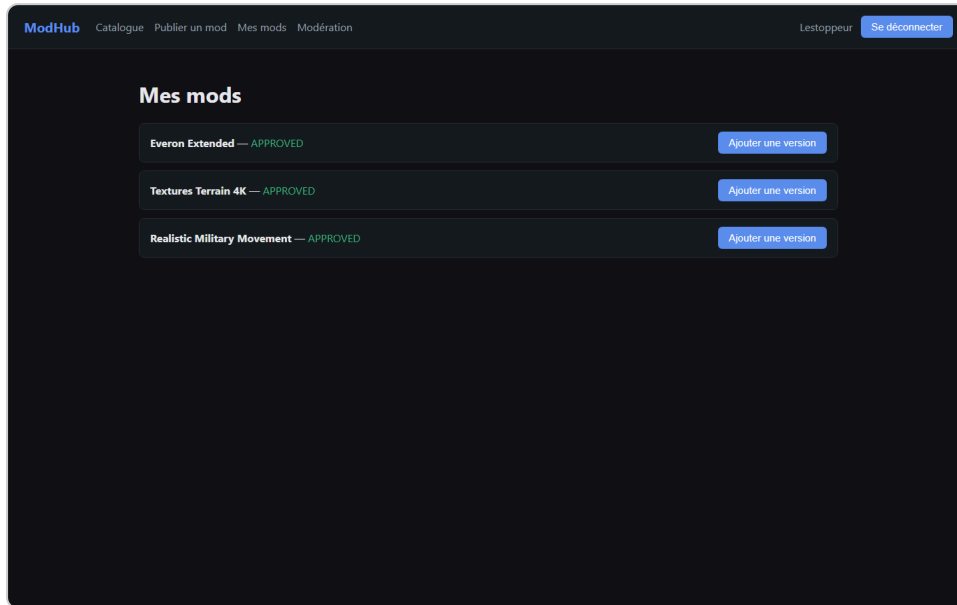
Recherche dynamique, filtrage côté client

La fiche mod

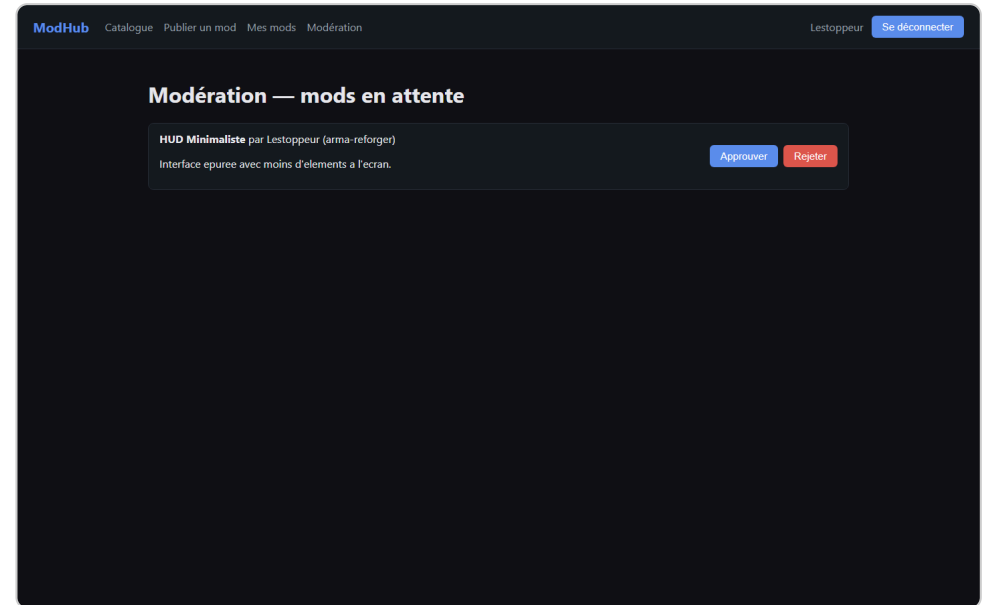


Données relationnelles + métadonnées NoSQL assemblées

Publication et modération



Espace créateur



File de modération admin

Vue d'ensemble

Frontend

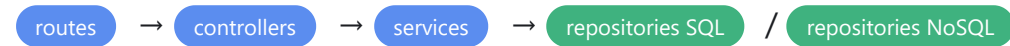
React 19 + TypeScript (Vite), React Router,
contexte d'authentification

Backend

Node.js + Express 5 + TypeScript, architecture
en couches

Données

PostgreSQL (Prisma) + MongoDB (Mongoose)



Pourquoi deux bases de données ?

PostgreSQL	MongoDB
Utilisateurs, mods, versions, notes, follows	Fil d'activité (forme variable par événement)
Relations fortes, intégrité référentielle	Métadonnées spécifiques à chaque jeu (schéma libre)

Maquette → statique → dynamique

- **Maquette** — wireframes basse fidélité des écrans clés avant implémentation
- **Statique** — CSS Grid / Flexbox, jetons de couleur centralisés, sans framework externe
- **Dynamique** — recherche avec debounce, contexte d'auth React, routes protégées par rôle

SQL et NoSQL, un vrai usage double

La création d'un mod illustre la collaboration des deux bases :

1. Écriture de l'entité `Mod` en PostgreSQL (Prisma)
2. Écriture des métadonnées spécifiques au jeu en MongoDB
3. Journalisation de l'événement dans le fil d'activité MongoDB

Vérifié par un test d'intégration qui contrôle les deux bases après l'appel API.

Mesures appliquées

- Mots de passe hachés (bcrypt), message d'erreur générique anti-énumération
- Autorisation vérifiée côté service (propriété, rôle admin)
- Erreurs centralisées : pas de fuite de stack trace au client
- Rate limiting sur l'authentification, CORS restreint
- Row Level Security activée sur les tables Supabase

Tests d'intégration bout-en-bout

Les tests exercent l'application réelle contre les vraies bases de développement (pas de mocks) pour valider un comportement effectif plutôt qu'une simulation.

9 / 9 tests passent — auth + cycle de vie complet d'un mod (SQL + NoSQL)

Difficultés rencontrées

- Pas de Docker fonctionnel en local → bascule vers Postgres managé (Supabase)
- Changement d'API majeur dans Prisma 7 (driver adapters) → vérification directe plutôt qu'hypothèse
- Typage strict Express 5 sur les paramètres de route → garde-fou dédié

Perspectives

- Upload de fichiers réel pour les versions de mods
- Notifications temps réel (WebSocket) sur le fil d'activité
- Tags libres en complément des catégories
- Tests end-to-end côté interface (Playwright)

Merci de votre attention

Dépôt du projet : github.com/Maximus04200/ModHub

Questions ?